

Session 3 · Deck 3.A2

Value Function Iteration: The First Complete Crossing

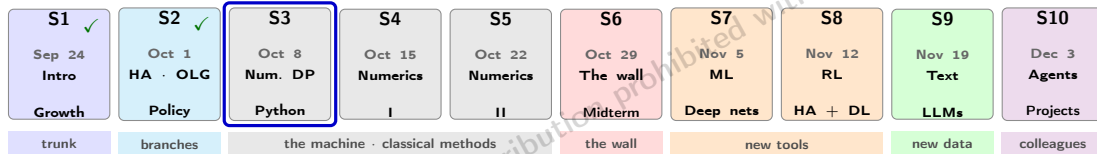
Zhigang Feng

Thursday, October 8, 2026

Build it on paper → solve it on the machine → push it to the frontier with AI

Where We Are

you are here: 3.A2 ▼



Build it on paper → solve it on the machine → push it to the frontier with AI

Block A: 3.A1 what it means to solve a model · 3.A2 value function iteration on a grid · 3.A3 Euler equations and time iteration · 3.A4 the worked example, end to end.

Twenty Lines, and the Number They Have to Hit

```
import numpy as np
alpha, beta, tol = 0.36, 0.95, 1e-6
k = np.linspace(0.05, 0.60, 200)

C = (k**alpha)[: , None] - k[None , :]
U = np.full_like(C, -1e10)
np.log(C, out=U, where=C > 0)

V, n = np.zeros(200), 0
while True:
    M = U + beta * V[None, :]
    Vn, pol = M.max(axis=1), M.argmax(axis=1)
    d = np.max(np.abs(Vn - V))
    V, n = Vn, n + 1
    if d < tol:
        break

g = k[pol]
g_star = alpha * beta * k**alpha
print(n, np.max(np.abs(g / g_star - 1)))
```

Session 1 promised you this in 25 lines. It takes twenty. Every one of the six decisions of 3.A1 is in there, each made the crudest way available.

272 0.00988

272 sweeps, and a policy that is wrong by up to 1%.

The rest of this deck is two questions: **where did the 1% come from**, and **why 272**?

What Those Lines Actually Do

1. Build the grid $\{k_i\}_{i=1}^N$ and guess v_0 .
2. Build the **payoff matrix** once, outside the loop:

$$U_{ij} = u(zk_i^\alpha - k_j), \quad -\infty \text{ if } c \leq 0.$$

3. Sweep: for every i ,

$$v_{n+1}(k_i) = \max_j \left\{ U_{ij} + \beta \sum_m \Pi_{jm} v_n(k_j, z_m) \right\}.$$

4. Stop when $\|v_{n+1} - v_n\|_\infty < \varepsilon$; read the policy off the **argmax**.

The one trick worth naming

U does not depend on v , so it is built **once** and reused 272 times. Infeasible cells are set to -10^{10} rather than tested inside the loop.

The max runs over *grid points*, so the policy is forced onto the grid. That single decision is the source of most of the 1%.

Algorithm and implementation steps adapted from QM Lec. 7, "VFI: Algorithm Structure" and "VFI: Implementation Steps".

Decision 2: Which Points, and How Many?

Uniform. $k_i = a + (i - 1) \frac{b-a}{N-1}$. Simple, and interpolation error is uniform in k — which is the wrong thing to make uniform, because v is far more curved at low k .

Logarithmic. $k_i = a + (b - a) \frac{e^{\theta(i-1)/(N-1)} - 1}{e^\theta - 1}$, with $\theta \in [2, 5]$: points bunched where the function bends.

Both cost the same to evaluate. The second is almost always better, and almost always forgotten.

Grid families from QM Lec. 7, “Grid Construction”. The curvature parameter is written θ here; the source calls it λ , which this course reserves for the invariant distribution.

Two boundary mistakes

Too narrow and the policy hits the top of the grid: $g(k_N) = k_N$ is a censored answer, not a solution.

Too wide and most of your points sit where the model never visits, paying $O(N^2)$ for nothing.

Always print $\max_i g(k_i)/k_N$. If it is 1, widen the grid before you believe anything.

Decision 3: Off the Grid, Straight Lines Only

Between two grid points,

$$\hat{v}(k) = v_i + \frac{v_{i+1} - v_i}{k_{i+1} - k_i} (k - k_i).$$

- Error is $O(h^2)$, with h the spacing.
- It **preserves monotonicity and concavity** — so a concave v stays concave, and the maximization stays well behaved.
- Its derivative is discontinuous at every node. Anything that differentiates $\hat{v}$ will feel those kinks.

Why not something smoother?

Cubic splines are $O(h^4)$ and C^2 — and they **overshoot**, which can destroy concavity and send the maximizer somewhere absurd.

Shape preservation beats formal accuracy inside a fixed-point loop. We stay linear today; the shape-preserving alternatives are built next week.

Adapted from QM Lec. 7, “Piecewise Linear Interpolation” and “Interpolation: Comparison”.

Decision 6: One Theorem, Used Two Ways

The contraction theorem gives *two* inequalities, and they answer different questions.

How many iterations? From v_0 ,

$$\|v_n - v^*\| \leq \frac{\beta^n}{1 - \beta} \|v_1 - v_0\| \Rightarrow n \gtrsim \frac{\ln \varepsilon}{\ln \beta}.$$

How wrong am I now? From the last step,

$$\|v_n - v^*\| \leq \frac{\beta}{1 - \beta} \|v_n - v_{n-1}\|.$$

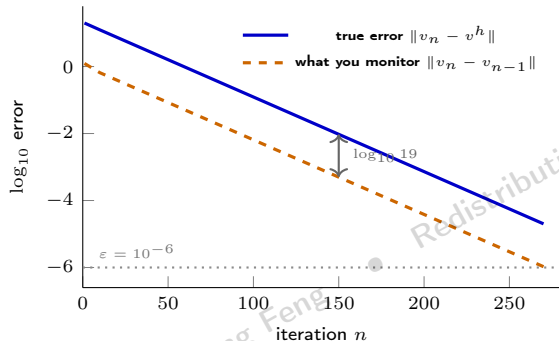
At $\beta = 0.95$ that factor is **19**.

$\beta^n / (1 - \beta)$ from the contraction mapping theorem as stated in QM Lec. 2; the one-step form is 1.B2. Measured counts at $\varepsilon = 10^{-6}$, $N = 200$.

β	$\ln \varepsilon / \ln \beta$	measured	$\frac{\beta}{1 - \beta}$
0.90	131.1	133	9
0.95	269.3	272	19
0.99	1374.6	1379	99

The count is a property of β alone — not of the grid, not of the model. Refining from $N = 50$ to $N = 800$ changed the iteration count by **zero**.

The Stopping Rule Is Not the Error



The two lines are **parallel**, separated by exactly $\log_{10} 19$. The quantity you watch is always a constant factor below the quantity you care about.

At $n = 272$, when the loop stops:

$\ v_n - v_{n-1}\ $	9.6×10^{-7}
guaranteed bound, $19\times$	1.81×10^{-5}
true error	1.81×10^{-5}

The $19\times$ bound is not pessimistic here — it is **attained**. Ask for 10^{-6} and you get 1.8×10^{-5} .

And You Can Do Much Better, for Free

Let $d_{\min} = \min_i (v_{n+1} - v_n)(k_i)$ and $d_{\max}$ the maximum. Then *both* sides are bounded:

$$v_{n+1} + \frac{\beta}{1-\beta} d_{\min} \leq v^* \leq v_{n+1} + \frac{\beta}{1-\beta} d_{\max}.$$

The naive rule throws away $d_{\min}$ and uses $\|\cdot\|_{\infty}$ for both. Keeping both ends costs two extra reductions per sweep and gives a **bracket** instead of a bound — take its midpoint and you have a free accuracy upgrade.

At $n = 272$ that bracket is 2.7×10^{-13} wide, against a true error of 1.8×10^{-5} .

Read that honestly

Eight orders of magnitude is *this model* flattering us: with $\delta = 1$ and log utility the increment $v_{n+1} - v_n$ is nearly constant in k , so $d_{\max} \approx d_{\min}$.

In general the bracket is narrower, not necessarily *that* much narrower. But it is never worse than the one-sided bound, and it is two lines of code.

MacQueen (1966), Porteus (1971). Statement adapted from QM Lec. 8, "MacQueen–Porteus Bounds".

What It Costs

Each sweep compares every grid point against every candidate choice: $O(N^2)$ per iteration, $O(N^2 N_z^2)$ with a shock. The iteration count is $O(\ln \varepsilon / \ln \beta)$ and does not depend on N . So the total is

$$\underbrace{N^2}_{\text{grid}} \times \underbrace{\frac{\ln \varepsilon}{\ln \beta}}_{\text{patience}}.$$

N	sweeps	comparisons
200	272	10.9 million
400	272	43.5 million
800	272	174 million

Two ways this becomes fatal

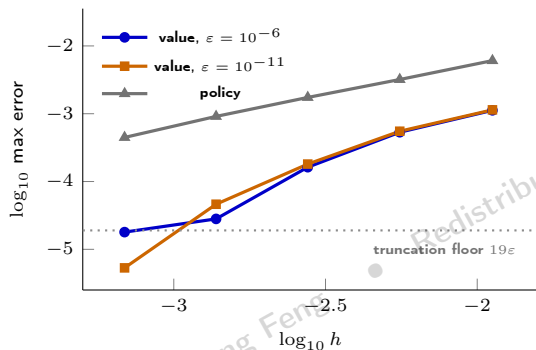
Patience. At a quarterly $\beta = 0.99$ the sweep count is 1,379 instead of 272 — a $5\times$ bill for changing the calendar.

Dimension. A second continuous state makes the grid N^2 and the sweep N^4 . 200 points per dimension is 1.6×10^9 comparisons *per sweep*.

Nothing here is wasted motion — it is the crude choices being paid for. Every one has a cheaper replacement, and that is next week.

Cost accounting adapted from QM Lec. 8, "Recap: Standard VFI Algorithm" and "Computational Bottlenecks".

Refining the Grid — Until It Stops Helping

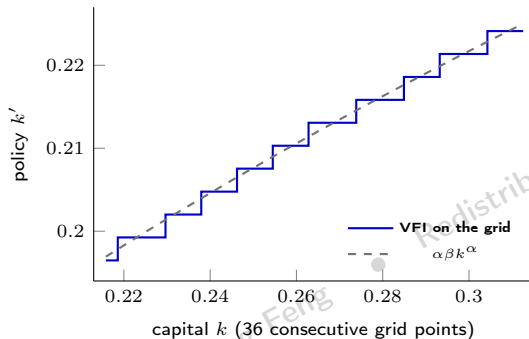


Santos & Vigo-Aguiar (1998): the value error is $O(h^2)$ and the policy error $O(h)$. Measured slopes at $\epsilon = 10^{-11}$: **1.89** and **0.93**.

At $\epsilon = 10^{-6}$ the value curve **flattens**. Going from $N = 400$ to $N = 800$ — four times the work — moves the value error from 2.8×10^{-5} to 1.8×10^{-5} , because it has hit the 19ϵ floor from 3.A1.

The budgets compose. Refining a grid while the tolerance is loose buys nothing, and you would never see it without running both.

Graded Against the Closed Form



Zoom in and the agreement becomes a **staircase**. Across these 36 states the computed policy takes only **11 distinct values** — it can return nothing but a grid point.

$\max v - v^* $	1.6×10^{-4}
max relative policy error	0.99%
grid spacing as a share of k'	$\approx 1.4\%$

The error is a **sawtooth**, not noise, and it does not shrink with more iterations. It is decision 4 — searching the grid — and no amount of patience will fix it.

Decision 5: Where Does the Expectation Come From?

Add productivity $z \in \{z_1, \dots, z_{N_z}\}$ with transition matrix Π . The sweep becomes

$$v_{n+1}(k_i, z_j) = \max_l \left\{ U_{ijl} + \beta \sum_m \Pi_{jm} v_n(k_l, z_m) \right\},$$

and the expectation is a matrix product: $EV = V @ \Pi$. T, computed once per sweep rather than inside the maximization.

With $z \in \{0.95, 1.05\}$ and Π symmetric with persistence 0.9, the closed form still holds state by state — so we can still grade it.

But where did Π come from?

We wrote it down. Real work starts from an estimated AR(1), $\ln z' = \rho \ln z + \sigma \epsilon'$, and must turn it into a chain that matches the persistence and the variance.

That is a genuine gap, not a detail: the two standard constructions disagree badly when $\rho > 0.95$, which is exactly the range macro uses. We build both next week.

Stochastic sweep adapted from QM Lec. 7, “Time Iteration with Stochastic Shocks” and the Lec. 8 recap; the chain used here is the one introduced in 1.B1.

Six Crude Choices, Six Gaps

What we did	What it cost	What we need	
Searched the grid for the max	the 1% sawtooth; $O(N^2)$ per sweep	optimization on an interval	S4
Linear interpolation	$O(h^2)$, kinked derivatives	approximation theory	S4
Wrote II down by hand	no link to any estimated process	discretizing an AR(1)	S4
Naive stopping rule	19× the error you asked for	two-sided bounds	today
$N^2 \times 272$ comparisons	174 million at $N = 800$	acceleration, better operators	S4
One state variable	nothing yet — and that is the problem	perturbation, projection	S5

Every row is a numerical technique that exists **because** a crude choice hurt. That is the honest reason the next two sessions look like a mathematics course: each tool on the list was invented to pay off one of these lines.

Takeaways

- Twenty lines take you from the Bellman equation to a policy function. The crossing is **not** the hard part — knowing how wrong you are is.
- The iteration count belongs to β alone: 272 sweeps at $\beta = 0.95$, 1,379 at 0.99, whatever the grid. The **accuracy** belongs to the grid.
- The quantity you monitor, $\|v_n - v_{n-1}\|$, sits a factor $\beta/(1 - \beta)$ below the error you care about — $19\times$ here, and that bound was **attained**, not merely respected.
- Error budgets compose. At $\varepsilon = 10^{-6}$, quadrupling the grid bought nothing, because the tolerance had already put a floor under the value error.
- The 1% policy error is a **sawtooth**, not noise: $\arg \max$ can only return a grid point. Diagnose the shape of an error, not just its size.

Bridge: The Same Model, Down the Other Road

Value function iteration never used the first-order condition. It maximized, brutally, at every point of every sweep — and the price was a policy quantized to the grid.

But we know the optimum satisfies

$$u'(c) = \beta \mathbb{E}[u'(c')R(k', z')].$$

That is an equation the policy must satisfy *exactly*, and solving an equation is a different — often cheaper, often sharper — numerical problem than searching for a maximum.

Next (3.A3)

Iterate on the **policy** instead of the value: the Coleman and Greenwood–Huffman operators, one root-find per grid point.

Then the comparison that matters — two methods, one model, measured against the same closed form — and the list of what still breaks when the model gets harder.